Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Name : Abhishek Pandey SAP ID : 60019220110

Roll No : B073 BATCH : B-2

Code :

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn import datasets

from mpl_toolkits.mplot3d import Axes3D


# -------------------------------------------------

# 1. Load Iris Dataset (3 classes)

# -------------------------------------------------

iris = datasets.load_iris()

X = iris.data[:, :3]   # Take first 3 features → 3D

y = iris.target


classes = np.unique(y)


# -------------------------------------------------

# 2. Compute Mean Vectors

# -------------------------------------------------

means = {}

for c in classes:

    means[c] = np.mean(X[y == c], axis=0)
```

```
    print(f"Mean of class {c}:\n", means[c])

overall_mean = np.mean(X, axis=0)

# -------------------------------------------------
# 3. Compute Within-Class Scatter Matrix (Sw)
# -------------------------------------------------
Sw = np.zeros((X.shape[1], X.shape[1]))

for c in classes:
    Xc = X[y == c]
    mean_vec = means[c].reshape(3,1)
    Sc = np.zeros((3,3))
    for x in Xc:
        x = x.reshape(3,1)
        Sc += (x - mean_vec).dot((x - mean_vec).T)
    Sw += Sc
    print(f"\nScatter matrix S{c}:\n", Sc)

print("\nWithin-class Scatter Matrix Sw:\n", Sw)

# -------------------------------------------------
# 4. Compute Between-Class Scatter Matrix (Sb)
# -------------------------------------------------
Sb = np.zeros((3,3))

for c in classes:
    n = X[y == c].shape[0]
```

```
    mean_diff = (means[c] - overall_mean).reshape(3,1)
    Sb += n * (mean_diff).dot(mean_diff.T)

print("\nBetween-class Scatter Matrix Sb:\n", Sb)

# ------------------------------------------------
# 5. Solve Eigenvalue Problem
# ------------------------------------------------
eig_vals, eig_vecs = np.linalg.eig(np.linalg.inv(Sw).dot(Sb))

print("\nEigenvalues:\n", eig_vals)
print("\nEigenvectors:\n", eig_vecs)

# ------------------------------------------------
# 6. Sort Eigenvectors
# ------------------------------------------------
idx = eig_vals.argsort()[::-1]
eig_vals = eig_vals[idx]
eig_vecs = eig_vecs[:, idx]

# Take top 2 eigenvectors (C-1 = 2)
W = eig_vecs[:, :2]

# ------------------------------------------------
# 7. Project Data (3D → 2D)
# ------------------------------------------------
X_lda = X.dot(W)
```

```python
# ------------------------------------------------
# 8. Visualization
# ------------------------------------------------

# ---- 3D Original Data ----
fig = plt.figure(figsize=(12,5))

ax = fig.add_subplot(121, projection='3d')
for c in classes:
    ax.scatter(X[y==c,0], X[y==c,1], X[y==c,2], label=f"Class {c}")
ax.set_title("Original 3D Data")
ax.set_xlabel("Feature 1")
ax.set_ylabel("Feature 2")
ax.set_zlabel("Feature 3")
ax.legend()

# ---- 2D LDA Projection ----
ax2 = fig.add_subplot(122)
for c in classes:
    ax2.scatter(X_lda[y==c,0], X_lda[y==c,1], label=f"Class {c}")
ax2.set_title("LDA Projection (2D)")
ax2.set_xlabel("LD1")
ax2.set_ylabel("LD2")
ax2.legend()

plt.tight_layout()
plt.show()
```

Result :

Mean of class 0:

[5.006 3.428 1.462]

Mean of class 1:

[5.936 2.77 4.26]

Mean of class 2:

[6.588 2.974 5.552]

Scatter matrix S0:

[[6.0882 4.8616 0.8014]

[4.8616 7.0408 0.5732]

[0.8014 0.5732 1.4778]]

Scatter matrix S1:

[[13.0552 4.174   8.962]

[4.174   4.825   4.05]

[8.962   4.05   10.82]]

Scatter matrix S2:

[[19.8128 4.5944 14.8612]

[4.5944 5.0962 3.4976]

[14.8612 3.4976 14.9248]]

Within-class Scatter Matrix Sw:

[[38.9562 13.63   24.6246]

[13.63   16.962   8.1208]

[24.6246 8.1208 27.2226]]

Between-class Scatter Matrix Sb:

[[63.21213333 -19.95266667 165.2484]

[-19.95266667 11.34493333 -57.2396]

[165.2484 -57.2396 437.1028]]

Eigenvalues:

[2.52943749e+01 -1.71509381e-15 2.05574943e-01]

Eigenvectors:

[[-0.34065015 -0.85073754 -0.19926757]

[-0.26903413 0.37284429 0.95830537]

[0.9008763 0.37044941 0.20480052]]